

Topic 1.8

1.8: Structures

It often happens that we have several pieces of related data about one object that we wish to keep together. For example, in keeping track of its employees, a large company typically maintains a phonebook that contains employee name, location, phone extensions, fax, etc.

The *structure* is the mechanism for collecting heterogeneous data about one object in one place. The prototypical example is

```
struct date {  
    int year;  
    int month;  
    int day;  
} today;
```

which declares `today` as a variable with three fields or components: `year`, `month`, and `day`, all of which happen to be of type `int`.



To access fields of a struct variable, use the dot `.` operator:

```
today.year = 1994;  
today.month = 6;  
today.day = 25;
```

Once accessed using the dot operator, each component behaves as an ordinary variable, which can be read, written, tested, etc.

In the preceding declaration, `date` is a tag, which we may later use to declare other variables of the same type:

```
struct date tomorrow;
```

Variables of the same structure type may be assigned to each other. A member by member copy occurs:

Assuming:

today	year 1994
	month 6
	day 25

the assignment

```
tomorrow = today;
```

has the following result:

tomorrow	year 1994
	month 6
	day 25

Note that if we have these declarations:

```
int x[100], y[100];
```

then the following assignment is illegal:

```
x = y;
```

However, using structures, we can write

```
struct int_array { int data[100]; };
int_array x, y;

x = y;
```

In this case, all the elements of *y.data* are assigned to corresponding elements of *x.data*.

If you have no need to declare variables of a structure type after the initial structure declaration, the tag may be omitted:

```
struct {
    int year;
    int month;
    int day;
} today;
```

This is uncommon, as you almost always need at least two variables of a given type.

It is common to define the structure in one declaration and define variables of that type in another declaration:

```
struct date{
    int year;
    int month;
    int day;
};

struct date today, tomorrow;
```

Structures may be initialized in their declarations:

```
struct date tomorrow = {1994, 6, 26};
```

In C++, tagnames (for structures, unions and bitfields) have the same force as typedef'd identifiers. Thus, in C++, we can omit the word *struct* and use only the tagname:

```
date vacation;
```

It is also quite useful to nest structures:

```
struct employee {
    char    name[40];
    char    ssn[9];
    struct date_of_birth;
};

struct employee me;
```

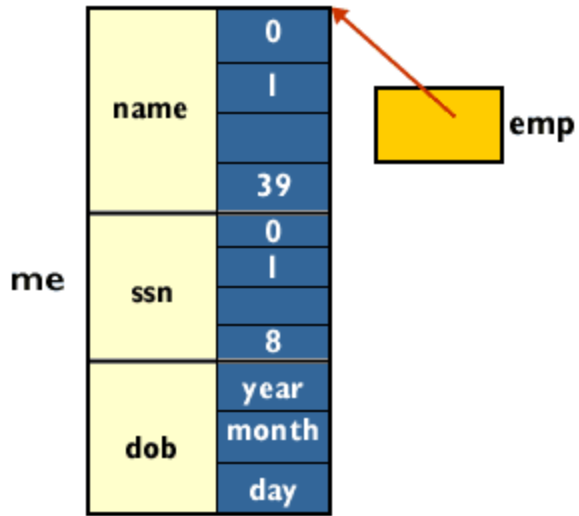


```
strcpy(me.name, "Blair, Gary J.");
me.date_of_birth.month = 8;
me.date_of_birth.day = 7;
```

We can use pointers to structures:

```
struct employee me;
struct employee *emp = &me; /* emp points at me. */
```

```
*emp.date_of_birth.day = 7; /* DOES NOT WORK! */  
(*emp).date_of_birth.day = 7; /* This does work. */  
emp->date_of_birth.day = 7; /* A shorthand that means the same. */.
```



to topic 1.9 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-1-dot-9>)